*DAUBES Ethan*

*DE BAYNAST Eloi*

*MARVIN Leo*

*VALAT Lou*

# Gadz'Air Project

# Design of a drone

***Submission date: 07/12/2025***

**Table of contents**


***1. INTRODUCTION*** ..... 4

***2. PROJECT BACKGROUND*** ..... 5

*2.1. General project objective* ..... 5

*2.2. Presentation of stakeholders* ..... 5

*2.3. Functional specifications provided by the client* ..... 5

*2.4. Work already done on the subject* ..... 5

***3. PROJECT ORGANIZATION*** ..... 6

*3.1. Definition of the planned schedule* ..... 6

***4. DEFINING THE SCOPE OF THE DRONE*** ..... 7

*4.1. Consolidated functional specifications* ..... 7

*4.1.1. Possible redefinition of the service function* ..... 10

*4.1.2. Search for additional constraints* ..... 10

***5. DRONE DESIGN*** ..... 11

*5.1. Determination of elementary solutions* ..... 11

*5.1.1. Determination of technical functions* ..... 11

*5.1.2. Choice of solutions* ..... 11

*5.2. Definition of the general architecture* ..... 12

*5.3. Definition of the structure* ..... 12

*5.4. Choice of components* ..... 13

*5.4.1. Flight controller* ..... 13

*5.4.2. Raspberry Pi 4* ..... 14

*5.4.3. The PA1010D/ADA4415 GPS module* ..... 15

*5.4.4. Drone arms* ..... 15

*5.5. Preliminary design of the artificial intelligence system* ..... 16

*5.5.1. Defining the objective of the AI* ..... 17

*5.5.2. Data research* ..... 17

*5.5.3. Presentation of the computer code used* ..... 18

*5.5.4. Presentation of the simulation system* ..... 18

*5.5.5. Conclusion* ..... 19

*5.6. 3D representation of the system* ..... 19

***6. FUNDING*** ..... 21

***7. IMPLEMENTATION*** ..... 22

*7.1. PROCUREMENT / PURCHASE OF COMPONENTS*.....22
*7.2. MANUFACTURING OF COMPONENTS*.....22
***8. CONCLUSION***.....23
***APPENDIX***.....24
***APPENDIX 1: SPECIFICATIONS PROVIDED BY DASSAULT***.....24
***APPENDIX 2: PYTHON CODE FOR DETECTION AND RECOGNITION***.....25
***APPENDIX 3: LIST OF COMPONENTS***.....27
***APPENDIX 4: COMPUTER CODE USED TO DECIDE WHETHER THE OBSTACLE IS DANGEROUS***.....28

# 1. Introduction

As part of the Dassault UAV Challenge competition, the Gadz'Air team is working on the design of a drone. This competition, now in its 12th edition, brings together students from different schools and invites them to design a drone with several autonomous features.

The purpose of this report is therefore to present the progress of the drone developed by the Gadz'Air team. The project spans a full year, with a first selection phase planned for December, followed by the final competition in May 2026.

In response to specifications set by Dassault, the Gadz'Air team, made up of Ethan Daubes, Eloi de Baynast, Leo Marvin and Lou Valat, has written a report that aims to address the following needs:

- Drawing up the specifications
- Proposing a drone architecture
- Sizing the required components
- Distribution of the work
- Provisional budget

# 2. Project background

## 2.1. General project objective

The objective is to build a drone in order to take part in the 12th edition of the DASSAULT UAV Challenge competition. Its function will be that of a drone with an integrated artificial intelligence able to carry out missions, such as image recognition and decision-making in order to avoid obstacles.

## 2.2. Presentation of stakeholders

The stakeholders are all the people and company departments involved in the project. They include:

- The project owner (client): the company Dassault. Dassault Aviation is a French aerospace company with recognized dual expertise in the manufacture of military aircraft and business jets.
- The project manager (prime contractor): the Gadz'Air team, a team of 4 students introduced in the introduction, students at the École Nationale Supérieure d'Arts et Métiers, Châlons-en-Champagne campus.

## 2.3. Functional specifications provided by the client

The specifications provided by the client are given in Appendix 1; however, the main elements can be summarized as follows:

1. UAV – Basic specifications:
    a. Compliance with DGAC regulations.
    b. Operational tracking system.
    c. Presence of a manual emergency stop switch ("kill switch").
    d. Operational manual mission-abort trigger.
    e. Loss of signal must under no circumstances result in the loss of the UAV.
2. Trajectory:
    a. The UAV must remain within the flight zone defined in the appendices at all times.
    b. The UAV must be able to follow trajectory A (appendices) within a maximum envelope of 3 meters.
3. Autonomy & Intelligence:
    a. The UAV must be able to follow a flight plan fully autonomously.
4. Project-related aspects:
    a. The total cost of the project must be below €1000.
    b. A detailed invoice of all components used (accurate to the cent) must be provided.
    c. A complete summary of the challenge must be included in the file.

## 2.4. Work already done on the subject

No work has yet been done on the subject; the Gadz'Air team is starting entirely from scratch.

# 3. Project organization

## 3.1. Definition of the planned schedule

The schedule below ends in January, when the second Arts et Métiers campus team will take over the project; however, the members registered for the competition will continue to supervise the project in order to carry it through to the competition. The numbers on the schedule correspond to the submission dates of our reports to our teachers, so that they can monitor our progress on the project; we only included the last 3 submissions. Finally, the letter D on the schedule corresponds to the submission of the report you are currently reading.

**PLANNING PRÉVISIONNEL PROJET DASSAULT**

| | | 2025 | | | | | | | | 2026 | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| MACRO-TACHES | TACHES | 45 | 46 | 47 | 48 | 49 | 50 | 51 | 52 | 1 | 2 | 3 |
| Propulsion | Choix moteurs | | | | | | | | | | | |
| Batterie | Bilan de conso. Éléctrique composants | | | | | | | | | | | |
| | Choix batterie de vol / composants | | | | | | | | | | | |
| | Achat matériel de recharge | | | | | | | | | | | |
| Structure | Référencement des pièces à imprimer | | | | | D | | | | | | |
| | Conception du modèle 3D adadpté aux attendus | | | | | | | | | | | |
| | Analyse RDM sur structure modélisée | | | | | | | | | | | |
| | Commande du PLA renforcé ou Alu. | | | | | | | | | | | |
| Télémétrie | Choix télécommande de contrôle de vol | | | | | | | | | | | |
| | Choix Antenne de connexion / Antenne vidéo | | | | | | | | | | | |
| IA | Implémentation et mise en place du retour vidéo | | | | | 3 | | | | | | |
| Vol autonome | Configuration carte IA | | | | | | | | | | | |
| | Configuration carte de contrôle de vol | | | | | | | | | | | |
| | Prise en main logiciel vol autonome | | | | | | | | | | | |
| | Configuration et installation du hardware | | | | | | | | | | | |
| Assemblage | Achat matériel | | | | | | | 4 | | | | |
| | Montage des composants sur structure | | | | | | | | | | | |
| | Ajout des composants éléctroniques | | | | | | | | | | | S |
| Réglementation | Formation DGAC | | | | | | | | | | | |
| Essais | Validation fonct. général et respect CDCF | | | | | | | | | | | |

## 4. Defining the scope of the drone

To define the precise needs of our system, as well as the components and programs required for it to work properly, we need to know our drone's flight conditions and, if possible, its environment.

Indeed, the project owner has told us that the competition will take place outdoors in a large open space and that the drone will not be exposed to a large number of natural obstacles.

However, we must take into account that on the day of the competition the weather may be poor (wind gusts, rain, storms…). These constraints, which are part of the system's environment, must therefore not be neglected.

### 4.1. Consolidated functional specifications

In order to best describe the objectives, expectations and desired criteria, a consolidated set of specifications is necessary. It makes the project easier for the stakeholders to understand.

Drawn from the competition's expectations and from the characteristics desired by our team, it summarizes the essential information needed to frame the project. The consolidated specifications are thus presented below :

| Sector | Criterion / Service function | Description | Requirement (value or criterion) | Detailed requirement | Priority | Technological solutions | Verification method | Remarks |
|---|---|---|---|---|---|---|---|---|
| Structure / Mechanics | Total mass | Take-off mass (MTOW) including battery and payload | | ≤ 2.5 kg (to be adjusted according to the competition category) | High | | Weighing on a scale | Comply with regulatory and competition limits |
| Structure / Mechanics | Frame rigidity | Resistance to vibrations and landing shocks | | No cracks after light impact tests | Medium | Solution: aluminium alloy AND/OR carbon fibre frame | Mechanical test + visual inspection | Discuss with a technician to help with the choice |
| Structure / Mechanics | Fasteners | Attachment system for propellers/motors and payloads | | Safety: nothing may come loose in flight | High | | Inspection + flight tests | |
| Propulsion | Motors | Type and number of motors | | Brushless, 4 motors (for a quadcopter) | High | | Datasheet check + bench test | |
| Propulsion | Propellers | Propeller size / pitch | | Motor/propeller match recommended by the datasheet | Medium | | Static thrust measurement | |
| Propulsion | Total thrust | Thrust available for climbing and maneuvers | | Maximum thrust > 2× weight in N (safety margin) | High | | Static thrust test | |
| Energy / Battery | Battery type | Chemistry, capacity and connectors | Target capacity 5000-12000 mAh | LiPo 4S/6S depending on motorization, | High | Lithium NCM OR Nickel / Cadmium battery | Spec check + endurance test | |
| Energy / Battery | Endurance | Flight time under mission conditions | Target: 20 min | ≥ 20 minutes in a real mission profile (with margin) | High | | Real flight test + telemetry log | |
| Energy / Battery | Battery safety | Protection against overcharge, short circuit and fire | | BMS or protection circuit + storage procedures | High | | Inspection, charge/discharge test | |
| Avionics / Sensors | Flight controller | Autopilot, GPS and IMU capability | | Controller with open-source firmware | High | Ardupilot | Configuration check + logs | Retrieve data remotely |
| Avionics / Sensors | GPS | Positioning for autonomous navigation | Accuracy of 2 m | RTK GPS if needed; accuracy < 2 m without RTK | Medium | | GPS accuracy test on the ground/in flight | https://docs.centipede.fr/docs/centipede/2_RTK.html |
| Avionics / Sensors | Additional sensors | Barometer, compass, optical (flow/vision) if necessary | | Barometer for altitude hold, optical flow for indoor use | Medium | | Functional tests in flight | |
| Telecommunications | Telemetry | Bidirectional telemetry link | | Useful range ≥ 1 km in open field | High | | Range test in open field | Requirement to be reviewed if needed |
| Telecommunications | Command link | Robustness of the RC link | | Frequency and redundancy according to the rules | High | | Latency and link test | |
| Software / Control | Flight modes | Manual, stabilized, autonomous (waypoints) | | Must support an autonomous flight plan and manual override | High | | Simulation test + real flight | |
| Software / Control | Algorithms | Guidance, navigation, obstacle avoidance | | Passive/active avoidance | Medium | | Simulation + field tests | |

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| Software / Control | Logs & Telemetry | Recording of flight parameters | | Complete logs (IMU, GPS, commands) during each flight | High | | Check of log files | Recording on a remote computer? |
| Payload | Camera / Mission sensors | Type of onboard sensor (RGB camera, multispectral, etc.) | | Compatible weight and power supply | Medium | | Integration test + image quality | |
| Payload | Carrying capacity | Ability to carry an object for the competition | 300 g | ≥ 300 g usable (example) | Medium | | Climb/flight test with load | |
| Safety / Regulations | Parachute / rescue system | Optional | | If fitted: deployment < 3 s | Low | | Ground test + procedures | |
| Safety / Regulations | Regulatory compliance | Local air rules and competition rules | | Compliance with mass limits, zones & frequencies | High | | Documentation check | |
| Safety / Regulations | Stop switch | Accessible motor cut-off | 1 s reaction | Kill switch accessible on the remote control + button on the drone | High | | Functional test | |
| Maintenance | Accessibility | Ease of disassembly and maintenance | | Replacement of an arm in < 15 min | Medium | | Demonstration test | |
| Maintenance | Documentation | Manuals, procedures and checklists | | Pre-flight and maintenance checklist available | High | | Existence of the documents | |
| Interface / Operation | Ground tools | Ground station for flight plan and telemetry | | Graphical interface for mission planning | Medium | | Usage test during flight | |
| Interface / Operation | Remote control ergonomics | Readability and access to critical controls | | Clear telemetry display + dedicated buttons | Medium | | Operator feedback | |
| Schedule & Budget | Project deadlines | Milestones: design, prototype, testing, delivery | 7 December | Design report in December; potential competition in May | High | | Gantt planning | |
| Schedule & Budget | Target cost | Total hardware budget | ≤ €1000 | | Medium | | Supplier quotes | |
| Procurement | Critical suppliers | Motors, ESCs, flight controller, batteries | See supplier details | | Medium | | Supplier list + lead times | |
| Reliability | Estimated MTBF | Mean time before failure | | MTBF > 100 hours for critical components | Medium | | Calculations + supplier feedback | |
| Performance | Maximum speed | Travel speed | $V_{max}$ = 19 $m.s^{-1}$ | ≥ 19 m/s peak (depending on mission) (around 70$kmh^{-1}$) | Medium | | Flight test | |
| Performance | Climb rate | Climbing capability | | ≥ 3 m/s | Medium | | Flight test | |
| Structure / Mechanics | Total mass | Take-off mass (MTOW) including battery and payload | | ≤ 2.5 kg (to be adjusted according to the competition category) | High | | Weighing on a calibrated scale | Comply with regulatory and competition limits |
| Structure / Mechanics | Frame rigidity | Resistance to vibrations and landing shocks | | No cracks after light impact tests | Medium | Solution: aluminium alloy AND/OR carbon fibre frame | Mechanical test + visual inspection | Discuss with a technician to help with the choice |
| Tests & Validation | Test benches | Motor, ESC and power supply tests | | Motor bench + recordings | High | | Test reports | |
| Tests & Validation | Flight campaign | Progressive test phases | | Indoor then outdoor sessions, increasing in complexity | High | | Flight log + logs | |

### *4.1.1. Possible redefinition of the service function*

The Gadz'Air drone will not only meet the prerequisites provided by the client; its service function will be autonomous flight with artificial intelligence. Its goal is to analyze, by itself, the obstacles perceived by the camera, identify and analyze them, then, with the help of a decision matrix and programs

### *4.1.2. Search for additional constraints*

As part of the drone's design, we had to strengthen the specifications that were provided as a guiding thread. We added numerical values following the criteria imposed by Dassault mentioned previously in the specifications, in order to quantify the objectives we set ourselves and see whether we have achieved them.

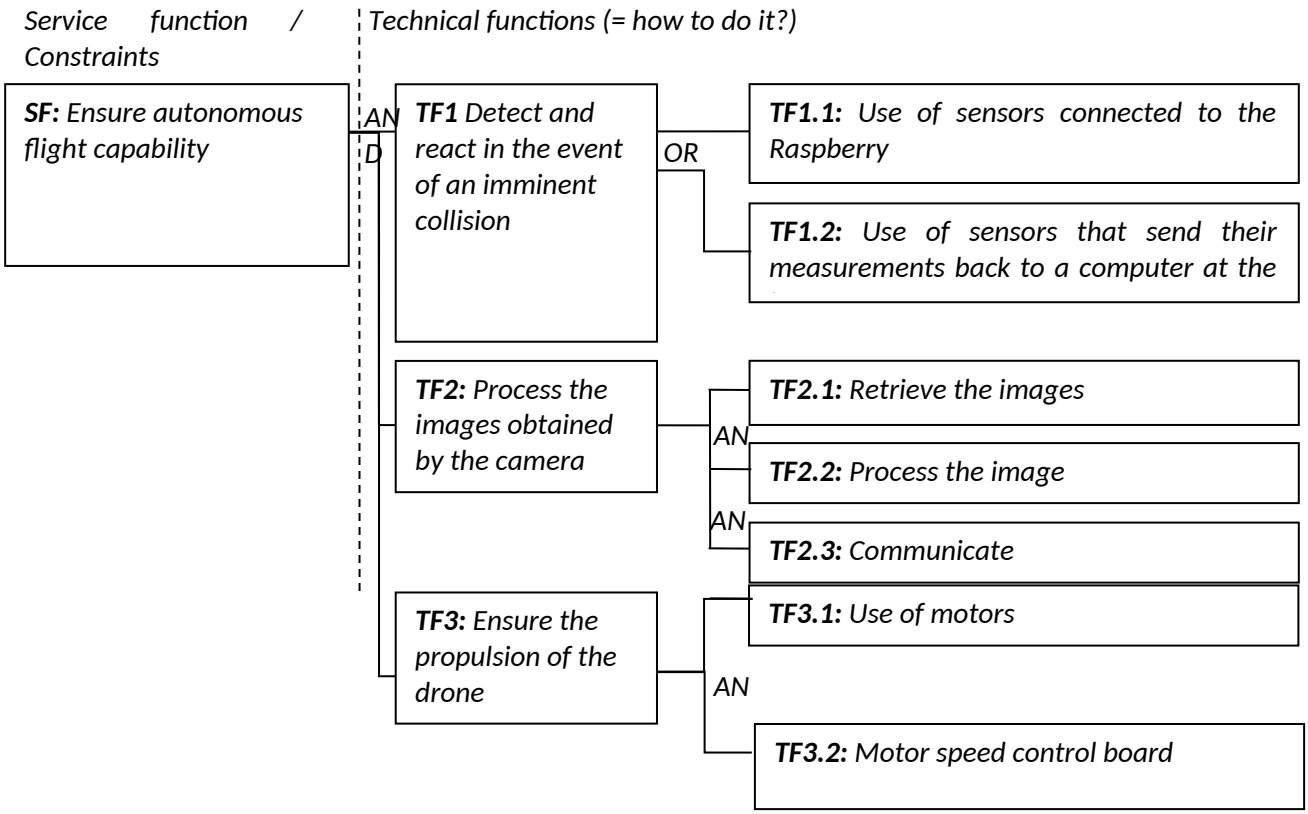
# 5. Drone design

For the competition, it is essential to know the drone's characteristics. Indeed, to meet the specifications, before assembling the various components we must be able to manufacture them, whether they are parts that will be physically mounted on the drone or computer code to be developed to ensure that the drone's mission is fulfilled.

## 5.1. Determination of elementary solutions

### 5.1.1. Determination of technical functions

In order to summarize the various technical functions performed by the drone, we will rely on a FAST (Function Analysis System Technique) diagram




### 5.1.2. Choice of solutions

#### 5.1.2.1. Definition and ranking of general selection criteria

From our specifications we can derive several selection criteria that will be decisive for the choices we will have to make among the technological solutions. These criteria are as follows:

- Flight time
- Shock resistance
- Total target cost below €1000
- Functionality (able to reach the performance set by the specifications)
- Mass

However, not all these criteria have the same importance. Our project makes component functionality a priority. In addition, since we are limited by the capacity of the motors, we must constantly seek to optimize mass

### *5.2. Definition of the general architecture*

Many options were considered for the drone. We first thought of a fixed-wing surveillance drone, and research and quick lift simulations were carried out. The initial goal was to build a “stealth” fixed-wing drone that could fly over areas at night and, using a thermal camera, carry out reconnaissance of a location and surveillance missions. However, due to several fairly complex elements such as automatic take-off and landing, the price of a high-definition infrared camera, and the danger of flying an aircraft that cannot be seen at night, the project was set aside. Our team therefore opted for a quadcopter coupled with artificial intelligence, because we considered it simpler and more in line with the future of drones. We followed the X500 model as the basis for the drone, as it is large enough to house an NVIDIA Orin Jetson Nano board, which was chosen to host our AI because of its price-to-computing-power ratio.

Our team has therefore chosen a quadcopter drone with 2 support legs to guarantee a soft landing and to keep the various electronic components away from the ground. The chassis will be made of 2 parallel plates, where we can place some of the components between the two plates and the NVIDIA board on the upper plate.

### *5.3. Definition of the structure*

The structure used will be that of the X500 drone, as it is large enough to house the AI board, and there is also enough room to fit a large number of components in a fairly modular way. The structure consists of four hollow cylindrical arms fixed by clamping at their ends. At the end of these arms there is room for the desired motors, and the wiring can run inside the arms. The central hub is made of 2 plates: a lower one that will house components such as the GPS, the flight computer and others. The upper plate will hold the AI board. The batteries will be fixed beneath the lower plate on the support provided for this purpose. In addition, the drone’s camera will be fixed on the mount on the side of the drone. Finally, 2 legs will provide the landing gear function of the drone.



*Figure 1 Image of the selected structure*

### *5.4. Choice of components*

#### *5.4.1. Flight controller*

The choice of flight controller fell on the RadioLink Pix6, because of its balance between reliability, performance and software compatibility. The board is a robust solution suited to an autonomous drone while keeping costs under control.

The RadioLink PIX6 is natively supported by ArduPilot, the reference software for autonomous flight. This guarantees:

- Immediate firmware integration,
- Reliable updates,
- The availability of many flight modes (Stabilize, Auto, Loiter, RTL, failsafes),
- Substantial community and documentation support, which allows in-depth training in handling the software.

**Suitable hardware architecture**

The board integrates redundant inertial sensors, a precise barometer and a stabilized power supply. Its standardized JST-GH connectors allow clean and quick wiring (GPS, telemetry, radio receiver, onboard computer). This reduces the risk of errors and ensures the reliability that is essential for autonomous missions.

**Scalability for future developments**

Thanks to its UART and USB ports, the RadioLink PIX6 allows the integration of a companion computer (e.g. Raspberry Pi) for onboard vision or advanced processing, thus meeting the needs of a project that may include AI.

**Cost/reliability ratio**

Positioned between hobby controllers and more expensive professional solutions, the RadioLink PIX6 offers a very good compromise for a project requiring precision and safety without exceeding the budget.

**Mission Planner** is the main tool for configuration, calibration and planning used with ArduPilot.
It was chosen for several reasons: Mission Planner automatically recognizes the board and makes it easy to install the firmware and configure the parameters without requiring advanced manipulation.

The software allows:

- The creation of complex trajectories using waypoints (key points),
- The definition of speeds, altitudes and specific actions,
- Simulation of the flight before execution.

This directly meets the autonomous flight requirements of the specifications.

Mission Planner offers:

- Display of sensors, batteries, motors and GPS,
- Recording of flight logs,
- Pre-flight check tools (failsafe, IMU redundancy, GPS quality),
- Post-flight analysis.

These elements are essential for the validation, tuning and reliability of the system.

Mission Planner communicates using MAVLink, a standard also used by the RadioLink PIX6 and by onboard computers (Raspberry Pi). This allows consistent integration of telemetry, long-range supervision and AI modules.

In conclusion, the **RadioLink PIX6**, combined with the **ArduPilot** software and the **Mission Planner**, constitutes a coherent, economical and technically suitable solution for an autonomous drone.
It offers:

- Sufficient hardware reliability,
- A complete software environment,
- Excellent scalability,
- Ease of integration compatible with the project's objectives.

### *5.4.2. Raspberry Pi 4*

The Raspberry Pi 4 was chosen as the onboard computer because it offers more than enough computing power to process the camera's video stream in real time, send images, handle any onboard processing and communicate efficiently with the Pixhawk flight controller. Its built-in interfaces (USB, UART, CSI for camera, Wi-Fi and Ethernet) allow simple and flexible integration while limiting the number of additional modules. This ability to process and transmit data continuously directly meets the drone's needs: real-time video feedback, mission monitoring and telemetry transmission.

Its compact size, reasonable power consumption and the availability of a large software ecosystem make it a reliable and scalable solution, particularly suited to a project requiring autonomy, remote supervision and the possibility of custom development. The Raspberry Pi 4 also makes it possible to clearly separate tasks: the Pixhawk handles piloting and safety (stabilization, RTL, flight

modes), while the Pi takes care of the “intelligent” part of the drone. This split improves the overall reliability of the system and aligns perfectly with the specifications of the autonomous drone.

### *5.4.3. The PA1010D/ADA4415 GPS module*

The PA1010D/ADA4415 GPS module was chosen to equip the drone because it offers good positioning accuracy and fast satellite acquisition thanks to its integrated ceramic antenna. Its multi-GNSS compatibility (GPS, Galileo, GLONASS) guarantees reliable localization, which is essential for autonomous flight-plan tracking, position hold and the Return To Launch function in the event of loss of the command link. Its compact size, low power consumption and UART interface make it easy to integrate with the Pixhawk flight controller, already selected in the drone’s electronic architecture.

The absence of a built-in compass is not a drawback; on the contrary, it allows the use of a separate external compass, less sensitive to the electromagnetic interference generated by the motors and power electronics. This GPS/compass separation is good practice in ArduPilot systems, improving heading reliability and flight stability. The PA1010D/ADA4415 thus meets the reliability, accuracy and compatibility criteria imposed by the specifications of the autonomous drone.

### *5.4.4. Drone arms*

As explained earlier in this report, the drone is based on a 4-arm structure (each carrying a motor) to provide its overall propulsion. Each arm will thus be subjected to various constraints resulting from the forces / torques generated by the motor located at its end. This gives the representation below:

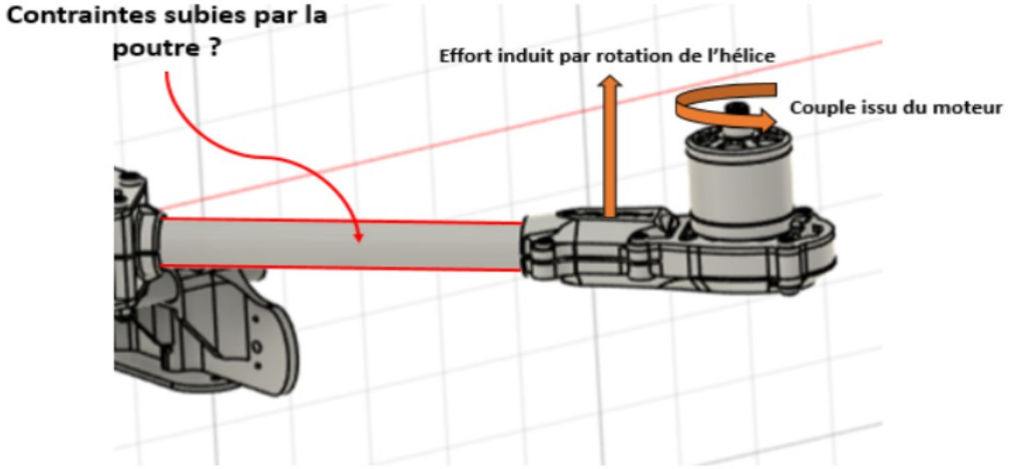



*Figure 2: Modeling of the situation studied*

As the main objective is the complete construction of the drone model under consideration, we must look at the materials used to build the various necessary structural components. For the arm, a Strength of Materials study is necessary to verify, according to the drone’s characteristics (dimensions / specifications…), whether a material is valid for making each arm of the drone.

The study will be structured as follows:

- Setting up the model for simulation and starting the calculation phase

- Setting the assumptions to simplify the model defined

- Applying classical or simulated solution methods to obtain results

- Conclusion on the choice of material based on the results obtained

At the current stage of the project, the model has been set up, and knowing the relevant data for the upcoming calculations now allows us to focus on making assumptions, in order to then carry out the pre-sizing calculations. For the moment, our thinking focuses on the following elements:

A - Arm treated as a beam (OK)

B - Inventory of loads (Choice of the type of loads for a simple application -> IN PROGRESS)

C – Choice of the calculation method to apply afterwards (TO COME).

| Sollicitation | N | T | $\mathcal{M}_t$ | $\mathcal{M}_f$ | Contraintes |
|---|---|---|---|---|---|
| Traction simple | $\neq 0$ | $=0$ | $=0$ | $=0$ | $\sigma_{11} \neq 0$ et $\overrightarrow{\tau} = 0$ |
| Cisaillement simple | $=0$ | $\neq 0$ | $=0$ | $=0$ | $\sigma_{11} = 0$ et $\overrightarrow{\tau} \neq 0$ |
| Torsion simple | $=0$ | $=0$ | $\neq 0$ | $=0$ | $\sigma_{11} = 0$ et $\overrightarrow{\tau} \neq 0$ |
| Flexion pure | $=0$ | $=0$ | $=0$ | $\neq 0$ | $\sigma_{11} \neq 0$ et $\overrightarrow{\tau} = 0$ |
| Flexion simple | $=0$ | $\neq 0$ | $=0$ | $\neq 0$ | $\sigma_{11} \neq 0$ et $\overrightarrow{\tau} \neq 0$ |
| Flexion composée | $\neq 0$ | $\neq 0$ | $=0$ | $\neq 0$ | $\sigma_{11} \neq 0$ et $\overrightarrow{\tau} \neq 0$ |
| Sollicitation composée | $\neq 0$ | $\neq 0$ | $\neq 0$ | $\neq 0$ | $\sigma_{11} \neq 0$ et $\overrightarrow{\tau} \neq 0$ |

*Figure 4: 3 List of possible loads applied to the beam*

Once these elements are set, we will be able to carry out the calculations and choose the suitable material for the arms of the final drone model. The drone will then be fully able to carry out the desired flight plan with arms that withstand the stresses generated by the motor in all circumstances.

### *5.5. Preliminary design of the artificial intelligence system*

In a context where artificial intelligence plays an increasingly important role, we chose to embrace this trend by designing a drone whose main characteristic is the integration of AI into the flight system, notably allowing it to fly autonomously once a mission is given. In addition, we are incorporating on our drone a system that processes the images captured by the drone's camera before they are sent back to the Home base via a computer.

#### *5.5.1. Defining the objective of the AI*

With the objective stated above, we will first seek to design the system that detects and classifies the various objects seen by the camera. We chose to start with this part because we need to be able to categorize obstacles before the drone has to decide whether or not the obstacle can be considered dangerous. Secondly, we use a system based on the difference between two successive

images to decide whether or not the obstacle is dangerous and to suggest a possible correction to avoid it.

### 5.5.2. *Data research*

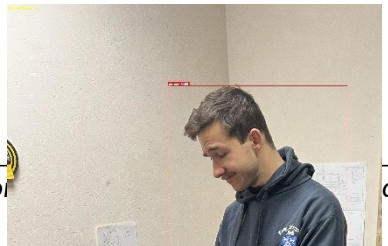
First, before we can begin designing our digital system, we need to consider the various possibilities. Our research led us to find several options, summarized in the following table:

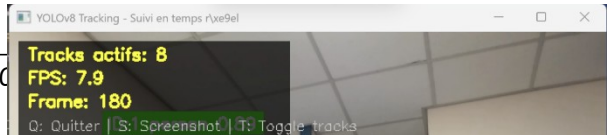
| Solution | Type | Advantages | Limitations | Application | Implementation |
|---|---|---|---|---|---|
| **OpenCV** | Classical vision (detection, tracking) | • Lightweight,<br>• Includes ready-made algorithms that can be used | • No Deep Learning | Target detection and tracking | Easy to implement on Raspberry Pi or Jetson Nano |
| **YOLO (ultralytics)** | Classical vision (detection, tracking) | • Good accuracy–speed compromise | • Does not run as well on Raspberry as on Jetson | Target detection and tracking | Optimized for Jetson |
| **PyTorch** | Deep Learning | • Can be coupled with other software thanks to its application programming interface (API) | • Needs data to train the model | Used to train a neural network to detect objects, classify them and navigate based on what the drone must accomplish | Develop the model on a computer, then export it to the Jetson |
| **TensorFlow** | Deep Learning with training and deployment on low-power systems | • Complete ecosystem | • Heavy system<br>• Complicated to get started with | • Detection/segmentation,<br>• Classification,<br>• Optimization of embedded models | Requires an adaptation layer for the Jetson or Raspberry board |

### 5.5.3. *Presentation of the computer code used*

We have attached in the appendix the Python code that retrieves the video and processes it to recognize its content, which will then be used to decide whether the obstacle is dangerous.

This code is based on the use of Yolov8n, a solution derived from the YOLO model available in Python. This model makes it possible to detect an object and classify it among a list of options. To detect objects, we call the model function and apply it to an image. This function returns the coordinates of the bounding rectangle of the various objects as well as the identifier of each object to classify it, and a confidence level for the identification.





In parallel, we have developed, using YOLO, a system to determine whether an obstacle should be considered dangerous. This code, presented in the appendix, relies on comparing two successive images. It produces an image in which the areas whose displacement is too large are highlighted and considered dangerous, because they are approaching too quickly.

This module, which we have just completed, has not yet been integrated into the drone, but it was first tested using the real-time transmission of a phone's video stream and offers promising prospects for our drone.

### *5.5.4. Presentation of the simulation system*

Before moving on to building the drone and commissioning the artificial intelligence, we chose to test our system digitally in order to reduce costs by avoiding building several drones.

To do so, we will use the AirSim software, which generates a simulation on Unreal Engine. AirSim is software developed by Microsoft that makes it possible to generate an environment in which we can run our artificial intelligence.

We have started simulating a virtual environment, in which we simulated a drone with our AI inside it, and we put it in front of situations that could happen in reality, and we see how it reacts depending on those situations; this will allow us to refine and train our AI. For example, avoiding objects on the requested trajectory.

### *5.5.5. Conclusion*

To summarize how the artificial intelligence system works, we propose a decision tree that presents the different steps.

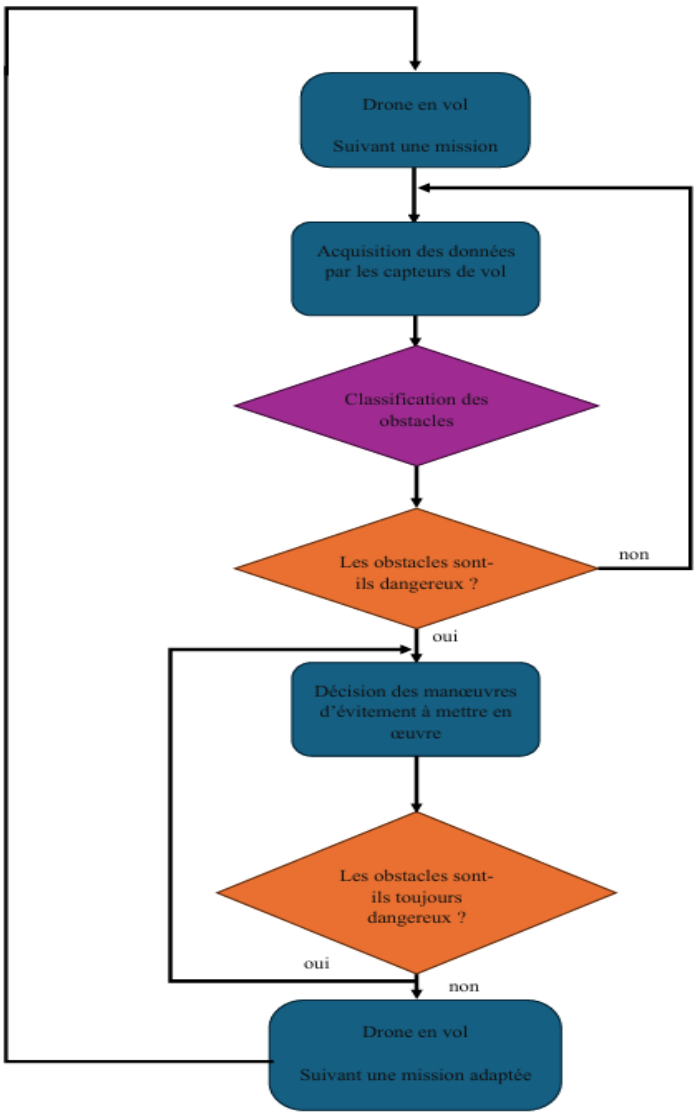



*Figure 6: Principle of incorporating the AI into the drone's operation*

In short, in organizing our work we chose to prioritize the design of the intelligence system, since we considered, after reflection and research, that this step was the most complicated and therefore deserved to be tackled first.

### *5.6. 3D representation of the system*

To show that the Gadz'Air team is able to rise to the challenge posed by this competition, we chose to propose an architecture of the drone we are seeking to build. To do so, we chose to propose a computer-aided design (CAD) representation of the system we are going to build.



*Figure 7: Proposed drone architecture*

## 6. Funding

Funding will amount to around €500 from the administration of the École Nationale Supérieure d'Arts et Métiers, Châlons-en-Champagne campus; then, should the Gadz'Air team be selected for the competition, an additional €500 of funding from the company Dassault Aviation. We present the provisional budget, which does not include the NVIDIA Orin Jetson Nano board (indeed, we chose for the moment to test the artificial intelligence modules with an NVIDIA Orin Jetson board lent to us by our supervising teacher, which cannot be mounted on a drone as it is too bulky, and a Raspberry Pi, to avoid too large an investment before deciding whether or not to keep the Jetson Nano):

| Élément | PRIX (EN EUROS) |
|---|---|
| Caméra | 21,8 |
| Radiolink PIX6 | 185,45 |
| Hélices | 25,4 |
| Chargeur de batterie | 36,3 |
| Raspberry pi 4 | 65,4 |
| Batterie | 44,33 |
| Moteurs*4 (ici prix totales) | 91,6 |
| LIGPOWER | 136,96 |
| Module GPS PA1010D ADA4415 ( pas de boussole) | 40,32 |
| télécommande émetteur + récepteur | 59,99 |
| | TOTAL |
| | 707,55 |

# 7. Implementation

To carry out flight and image-processing tests and simulations, it is necessary to obtain the essential elements for assembling the drone. To this end, this section aims to detail the purchases made since the beginning of the year.

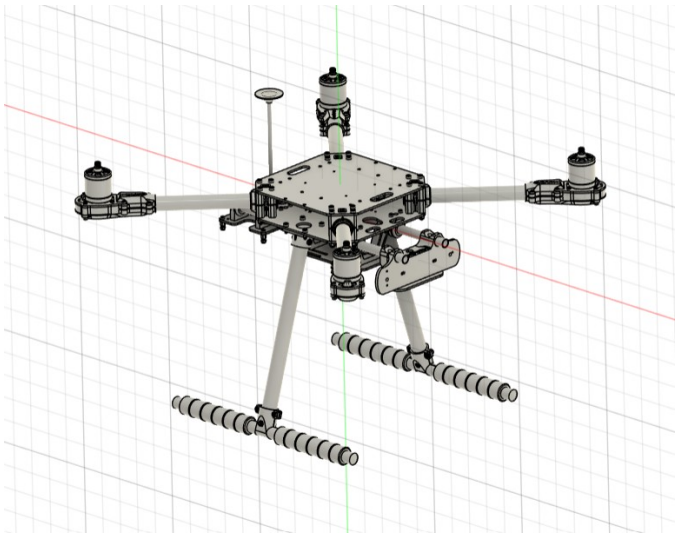
## 7.1. Procurement / purchase of components

To meet the needs of our drone and to support the proper operation of the AI and image processing, it is necessary to plan the purchase of components, since the École Nationale Supérieure d'Arts et Métiers in Châlons-en-Champagne does not allow component purchases once December has passed, apart from a few exceptions.

Having a partnership with the school, all purchases were made on the Robotshop website. The list of components purchased and obtained is given in Appendix 3.

## 7.2. Manufacturing of components

We must build a mock-up that will hold the components together and thus create the base of the drone. It was inspired by an existing mock-up.



The question now is which parts will be manufactured by which means and in which materials. First, the plates that will support the electronic components will be produced by sand casting and then machined with a three-axis milling machine.

As for the shafts that will support the motors, it is necessary to strengthen the structure with carbon-reinforced PLA filament or, given the problems PLA filament has with bad weather, it would be preferable to turn to a composite using a machine such as the MacForge Two.

The rest of the parts could simply be obtained by 3D printing with PLA filament, at least for the first mock-up.

## 8. Conclusion

In conclusion, the AI is under development and almost finished. The various modules of this AI are ready to be integrated into the drone and will soon be integrated into the AI over the coming weeks in December. As the team has managed to get to grips with the software and run flight simulations, the autonomous flight part with the ArduPilot software is under way.

In addition, the components have been purchased and are being delivered. All the drone's various structural parts are being printed.

# Appendix:

## Appendix 1: Specifications provided by Dassault

| Theme | Requirements | Level | Max. points | Notes | Actual points |
|---|---|---|---|---|---|
| UAV Basic Specifications | DGAC compliance | Mandatory | x | Test dd. | |
| | Tracking system is operational | Mandatory | x | Test dd. | |
| Safety | Manual emergency kill switch | Mandatory | x | Test dd. | |
| | Manual mission cancellation trigger is operational | Mandatory | x | Test dd. | |
| | In case of error, UAV automatically ends the mission | Important | 1 | | |
| | Ending the mission leads to RTH | Nice to have | 1 | | |
| | Ending the mission does not damage the drone | Nice to have | 1 | | |
| | Loss of signal does not lead to loss of UAV | Mandatory | x | Test dd. | |
| | Loss of signal triggers RTH automatically | Important | 1 | | |
| | Real-time battery monitoring | Important | 1 | | |
| | On ground station | Nice to have | 1 | | |
| | Mission can automatically be stopped once below a given battery capacity | Nice to have | 1 | | |
| Trajectory | UAV must stay within the flight area defined in section XX of the appendices at all time | Mandatory | x | This is mandatory at all times. | |
| | UAV can follow trajectory A from the appendices within a 3 meters envelop | Mandatory | x | Test dd. // Trajectory accuracy must be proven by the team to the jury | |
| | Within a 1 meter envelop | Important | 1 | 1. Trajectory accuracy must be proven by the team to the jury | |
| | Within a 50cm envelop | Nice to have | 1 | 1. Trajectory accuracy must be proven by the team to the jury | |
| | Within 1 minute | Important | 1 | | |
| | Within 30 seconds | Nice to have | 1 | | |
| | Quantify the uncertainty and performance of the positioning system | Important | 1 | | |
| Flight Time | UAV can fly for more than 15 minutes | Important | 1 | | |
| | UAV can fly for more than 30 minutes | Nice to have | 1 | | |
| | UAV can fly virtually indefinitely | Nice to have | 1 | | |
| Autonomousness & Intelligence | UAV can follow a flight plan autonomously | Mandatory | x | Test dd. | |
| | Along with a real time thorough monitoring on ground station | Important | 1 | Battery level, speed, heading, mission status, code progress, angles, etc. | |
| | Along with a real time video from the UAV | Nice to have | 1 | | |
| | Onboard computing | Nice to have | 1 | Ground station only monitor data but does not do any computing | |
| | UAV can guide itself by computer vision and/or pattern recognition | Nice to have | 10 | | |
| | GPS denied capacity using computer vision | Nice to have | 20 | | |
| | Complex pattern recognition and/or classification and/or any machine learning technique | Nice to have | 1 | | |
| | Relevant reinforcement learning | Nice to have | 10 | | |
| | Fixed wing UAV can take off autonomously | Nice to have | 2 | | |
| | Within less than 10 meters | Nice to have | 1 | | |
| | Vertically (VTOL) | Nice to have | 1 | | |
| | Capacity to automatically generate a flight plan just before the demonstration given uptown zone and for specific | Nice to have | 10 | | |
| | Autonomous in-flight reconfiguration capacity | Nice to have | 10 | | |
| Payload | UAV can carry 150g of payload | Important | 1 | UAV can fly with and without this payload. Its mass must be proven to the jury | |
| | UAV can carry 500gr of payload | Nice to have | 1 | UAV can fly with and without this payload. Its mass must be proven to the jury | |
| | UAV can air drop a payload on a point target within 10cm radius | Important | 1 | | |
| | Within a 5cm radius | Nice to have | 1 | | |
| | Without any bounce | Nice to have | 1 | | |
| | Air dropping is autonomous | Nice to have | 1 | | |
| | Air dropping triggered by pattern recognition | Nice to have | 1 | | |
| | Trajectory adaptation to drop the payload on the detected pattern | Nice to have | 10 | | |
| | Air dropping from a fixed wing UAV | Nice to have | 1 | | |
| Noise and surveillance | UAV can fly over a single point making less than 75dB over that period | Important | 1 | 1. 50 below 10 meters of altitude | |
| | UAV can fly over a point crossing less than 75dB over that period | Nice to have | 1 | 1. Trajectory TBD in appendices over a given area | |
| | UAV can capture ground images | Nice to have | 1 | Possible to read font 12 on a A4 piece of paper from above 10 meters of altitude | |
| | In real time | Nice to have | 1 | | |
| Connectivity | Preparing and sending a flight plan with a smartphone | Nice to have | 1 | | |
| | Dynamic interaction with a smartphone | Nice to have | 1 | | |
| | Getting accurate time-updated mission ground | Nice to have | 10 | | |
| Air Collaboration | Having a second autonomous flying platform in support of the master drone | Nice to have | 20 | | |
| | The secondary drone can follow the master drone | Nice to have | 10 | | |
| | The secondary drone can perform its own mission (in collaboration with the master drone) | Nice to have | 10 | | |
| | The secondary drone is dropped from the master drone | Nice to have | 10 | | |
| | Automatic mission scheduling taking advantages of each drone strength | Nice to have | 20 | | |
| Customization | Hand made assembly | Important | 1 | UAV is not a RTF drone | |
| | Custom flight control laws | Nice to have | 1 | 1. Depends on the complexity determined by the jury | |
| | Custom intelligence code and components | Nice to have | 1 | 1. Depends on the complexity determined by the jury | |
| | Custom electronics components | Nice to have | 1 | 1. Depends on the complexity determined by the jury | |
| | Any relevant smart tuning | Nice to have | 1 | 1. Depends on the complexity determined by the jury | |
| Project related | Total control the project below 1000€ | Mandatory | x | | |
| | Dassault's donation savings | Important | 10 | 10pts = 0€ from 0€, 0pt = 1000€ from 0€, linear decay | |
| | Team's expenses | Important | 10 | 10pts = 0€ from team, 0pt = 500€ from team, linear decay | |
| | Bill of all components used, cents precision | Mandatory | x | | |
| | Summary of all challenge | Mandatory | x | | |
| | Curriculum integration | Important | 1 | | |
| | Mixed sex team | Important | 10 | | |
| Spirit | Spirit as a whole | Important | 20 | Reading the rules, contesting the jury, negative attitude towards the other participants, punctuality, etc. | |
| | Innovation | Important | 10 | Innovations regarding the whole project | |
| | Ambition vs pragmatism | Important | 5 | Announcing 10 features but doing 1 of them correctly = BAD / Doing perfectly everything that has been promised = GREAT | |

# Algorithme de détection pour drone

MARVIN Leo

5 décembre 2025

## 1 Introduction

Ce document presente le fonctionnement d'un programme permettant de detecter des obstacles a l'aide d'une camera et de renvoyer une video mettant en valeur les objets detectes.

## 2 Principe général

Le systeme repose sur l'exploitation de la bibliotheque OpenCV compatible avec Python. Cette bibliotheque permet de pouvoir reconnaitre les differents objets parmi une liste de possibilites de 80 classes.

## 3 Code informatique

```
import cv2
from ultralytics import YOLO
import math
import random
import numpy as np

# ---------------- CONFIGURATION ----------------
# Charge le modele.
model = YOLO('yolov8n.pt')  # Modele pre-entraine sur COCO (80 classes)

# Source de la video
# Mettre 0 pour la webcam, le chemin d'un fichier "video.mp4",
# ou l'URL RTSP du drone (ex: "rtsp://192.168.1.1:554/stream")
#video_source = 0
video_source = "personne_test2.jpg"  # Image test

# Recuperer toutes les classes disponibles dans le modele
CLASS_NAMES = model.names  # Dictionnaire {id: nom_classe}
print(f"Modele charge avec {len(CLASS_NAMES)} classes disponibles")


# Generer une couleur unique pour chaque classe
def generate_colors(num_classes):
    colors = {}
    for class_id in range(num_classes):
        # Generer des couleurs vives et variees
```



```
        hue = (class_id * 137) % 360  # Utiliser l'angle d'or pour une
            bonne repartition
        saturation = 0.7 + random.random() * 0.3  # Entre 0.7 et 1.0
        value = 0.8 + random.random() * 0.2  # Entre 0.8 et 1.0

        # Convertir HSV vers BGR pour OpenCV
        hsv_color = cv2.cvtColor(
            np.uint8([[[hue / 2, saturation * 255, value * 255]]]),  #
                OpenCV HSV: H/2, S*255, V*255
            cv2.COLOR_HSV2BGR
        )
        colors[class_id] = tuple(map(int, hsv_color[0][0]))
    return colors


# Generer les couleurs pour toutes les classes
COLORS = generate_colors(len(CLASS_NAMES))

# ---------------- MAIN LOOP ----------------
import numpy as np

cap = cv2.VideoCapture(video_source)

if not cap.isOpened():
    print("Erreur : Impossible d'ouvrir le flux video.")
    exit()

while True:
    ret, frame = cap.read()
    if not ret:
        break

    # Faire la detection sur l'image
    results = model(frame, stream=True)

    # Traiter les resultats
    for r in results:
        boxes = r.boxes
        if boxes is not None:
            for box in boxes:
                # 1. Recuperer les coordonnees de la boite
                x1, y1, x2, y2 = box.xyxy[0]
                x1, y1, x2, y2 = int(x1), int(y1), int(x2), int(y2)

                # 2. Recuperer la classe detectee
                cls_id = int(box.cls[0])

                # 3. Recuperer le nom de la classe depuis le modele
                if cls_id in CLASS_NAMES:
                    class_name = CLASS_NAMES[cls_id]
                else:
                    class_name = f"Classe {cls_id}"

                # 4. Recuperer la couleur pour cette classe
                if cls_id in COLORS:
                    color = COLORS[cls_id]
                else:
                    color = (0, 0, 255)  # Rouge par defaut
```

```
            # 5. Recuperer le score de confiance
            conf = math.ceil((box.conf[0] * 100)) / 100

            # 6. Affichage
            # Dessiner le rectangle
            cv2.rectangle(frame, (x1, y1), (x2, y2), color, 2)

            # Creer le texte a afficher
            text_display = f"{class_name} {conf:.2f}"

            # Calculer la taille du texte pour le fond
            font = cv2.FONT_HERSHEY_SIMPLEX
            font_scale = 0.6
            thickness = 2
            text_size = cv2.getTextSize(text_display, font,
               font_scale, thickness)[0]

            # Dessiner le fond du texte
            text_bg_y1 = max(y1 - text_size[1] - 10, 0)
            cv2.rectangle(frame,
                          (x1, text_bg_y1),
                          (x1 + text_size[0] + 10, y1),
                          color,
                          -1)

            # Afficher le texte
            cv2.putText(frame,
                        text_display,
                        (x1 + 5, y1 - 5),
                        font,
                        font_scale,
                        (255, 255, 255),  # Texte en blanc
                        thickness)

    # Afficher le nombre de detections
    detection_count = len(boxes) if boxes is not None else 0
    cv2.putText(frame,
                f"Detections: {detection_count}",
                (10, 30),
                cv2.FONT_HERSHEY_SIMPLEX,
                0.8,
                (0, 255, 255),
                2)

    # Sauvegarder l'image traitee
    cv2.imwrite("Test_personne2_detected.jpg", frame)

    # Afficher la video/image
    cv2.imshow('YOLO - Detection de toutes les classes', frame)

    # Redimensionner la fenetre si l'image est trop grande
    height, width = frame.shape[:2]
    if width > 1280 or height > 720:
        frame_resized = cv2.resize(frame, (1280, 720))
        cv2.imshow('YOLO - Detection de toutes les classes',
           frame_resized)
```



```
    # Quitter avec la touche 'q'
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

# Liberer les ressources
cap.release()
cv2.destroyAllWindows()

# Afficher un resume des classes detectees
if boxes is not None:
    print("\n=== R\'ESUM\'E DES D\'ETECTIONS ===")
    unique_classes = {}
    for box in boxes:
        cls_id = int(box.cls[0])
        if cls_id in CLASS_NAMES:
            class_name = CLASS_NAMES[cls_id]
            if class_name not in unique_classes:
                unique_classes[class_name] = 0
            unique_classes[class_name] += 1

    for class_name, count in unique_classes.items():
        print(f"{class_name}: {count} detection(s)")

#Generation d'une seule image
    cv2.imwrite("Test_personne3.jpg", frame)


#Generation de la video
# frame_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
# frame_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
# fps = cap.get(cv2.CAP_PROP_FPS) # Images par seconde
#
# fourcc = cv2.VideoWriter_fourcc(*'mp4v')
#
# out = cv2.VideoWriter('video_traitee.mp4', fourcc, fps, (frame_width,
   frame_height))

#cap.release()
#cv2.destroyAllWindows()
```

Listing 1 – Capture de la video, detection des points d'interet et renvoi de la video annotee

*Appendix 3: List of components*



TENERGY

**Universal Smart Charger 1A for Li-Ion/Polymer Battery Pack (3.7V-14.8V 1-4 Cell)**

SKU: RB-Ten-28

**€36,30**

− 1 + Remove

− 1 + Remove

€36,30



LYNXMOTION

**11.1V, 3500mAh, 30C LiPo Battery**

SKU: RB-Ste-02

**€44,33**

− 1 + Remove

− 1 + Remove

€44,33



HQPROP

**10" x 4.5" Black Multi-Rotor CW Propeller Pair**

SKU: RB-Hqp-16

**€6,35**

This product is not available in the requested quantity. 3 of the items will be backordered.

3 of the items will be backordered.

− 4 + Remove

− 4 + Remove

€25,40



LIGPOWER

**LIGPOWER Air 40A 2-6S Multi-Rotor UAV Drone ESC**

SKU: RB-Tmo-31

**€34,24**

This product is not available in the requested quantity. 4 of the items will be backordered.

4 of the items will be backordered.

Re-stocking soon

− 4 + Remove

− 4 + Remove

€136,96



IFLIGHT-RC

**iFlight XING2 2207 FPV Motor Unibell 2755KV**

SKU: RB-Ifl-130

**€22,90**

This product is not available in the requested quantity. 4 of the items will be backordered.

4 of the items will be backordered.

On Demand

* Delays depend on the order frequency from each supplier and the approximate arrival date is indicated in the due date. Returns are not accepted on On Demand and Clearance Items except when they are found defective, in which case the product may be repaired or replaced at RobotShop's discretion.

− 4 + Remove

€91,60



SEEEDSTUDIO

**Raspberry PI GPS Module**

SKU: RB-See-570

**€40,32**

− 1 + Remove

− 1 + Remove

€40,32



RASPBERRY PI

**Raspberry Pi 4 Compute Module 4 - 16GB eMMC, 2GB LPDDR4 RAM (CM4002016)**

SKU: RB-See-1104

**€65,40**

− 1 + Remove

− 1 + Remove

€65,40



RADIOLINK ELECTRONIC

**RadioLink PIX6 Flight Controller**

SKU: RB-Rlk-56

**€185,45** ~~€218,17~~

Returns are not accepted on Special Order and Clearance Items except when they are found defective, in which case the product may be repaired or replaced at RobotShop's discretion.

− 1 + Remove

− 1 + Remove

€185,45



ARDUCAM

**ArduCam 8MP IMX219 Wide Angle Camera Module for NVIDIA Jetson Nano**

SKU: RB-Adu-219

**€21,80**

− 1 + Remove

− 1 + Remove

€21,80



RADIOLINK ELECTRONIC

**RadioLink T8FB(BT) 2.4GHz 8CH Transmitter w/ R8EF 8CH Receiver**

SKU: RB-Rlk-40

**€59,99**

− 1 + Remove

− 1 + Remove

€59,99

## Appendix 4: Computer code used to decide whether the obstacle is dangerous

```
            else:
                centroid_x_norm = 0.0

        # Collision decision (simple threshold logic)
        collision_alert = False
        if moving_pixels > CONFIG["movement_pixel_count_threshold"] and
           mean_mag > CONFIG["magnitude_threshold"]:
            collision_alert = True

        # Compute avoidance command with PID:
        # Error is centroid_x_norm (positive -> object to the right ->
           we want to yaw left, negative vice-versa)
        # We'll invert sign so positive error = need to yaw right.
        pid_error = -centroid_x_norm  # choose sign convention for
           control (tune as needed)
        yaw_cmd_raw = pid.update(pid_error, dt)

        # scale and clamp yaw command to [-max, max]
        max_yaw = CONFIG["max_yaw_command"]
        yaw_cmd = float(max(-max_yaw, min(max_yaw, yaw_cmd_raw)))

        # forward command reduction based on mean magnitude (if object
           close, reduce forward speed)
        # normalize mean_mag to [0..1] using an expected mag range
        # NOTE: tune these expected values for your camera/speed
        mag_expected_min = 0.0
        mag_expected_max = 10.0  # adjust according to your observations
        normalized_mag = normalize(mean_mag, mag_expected_min,
           mag_expected_max)
        # if collision alert -> strong braking, otherwise forward
           proportional to (1 - normalized_mag)
        if collision_alert:
            forward_cmd = float(-0.3)  # negative = slow/stop/backoff;
               tune with care
        else:
            forward_cmd = float(CONFIG["max_forward_command"] * (1.0 -
               normalized_mag))
            # clamp
            forward_cmd = max(-CONFIG["max_forward_command"], min(CONFIG
               ["max_forward_command"], forward_cmd))

        # smoothing
        yaw_history.append(yaw_cmd)
        forward_history.append(forward_cmd)
        yaw_cmd_s = float(np.mean(yaw_history))
        forward_cmd_s = float(np.mean(forward_history))

        # For debug / simulation: print and overlay
        info_text = f"mean_mag={mean_mag:.2f} med={median_mag:.2f}
           mov_px={moving_pixels} ALERT={collision_alert}"
        cmd_text = f"YAW={yaw_cmd_s:+.2f} FORWARD={forward_cmd_s:+.2f}"
        # draw ROI
        vis = frame.copy()
        cv2.rectangle(vis, (x1, y1), (x2, y2), (200, 200, 0), 2)
        # overlay mask scaled to full image for visualization
        mask_full = np.zeros_like(gray)
        mask_full[y1:y2, x1:x2] = mask
        mask_rgb = cv2.cvtColor(mask_full, cv2.COLOR_GRAY2BGR)
```



```
        vis = cv2.addWeighted(vis, 0.8, mask_rgb, 0.6, 0)

        # draw centroid
        if M is not None and M["m00"] != 0:
            cx_img = x1 + int((centroid_x_norm/2.0 + 0.5) * (x2 - x1))
            cy_img = y1 + int((0.5) * (y2 - y1))
            cv2.circle(vis, (cx_img, cy_img), 6, (0, 0, 255), -1)

        # print texts
        cv2.putText(vis, info_text, (10, 20), cv2.FONT_HERSHEY_SIMPLEX,
           0.55, (255, 255, 255), 1)
        cv2.putText(vis, cmd_text, (10, 45), cv2.FONT_HERSHEY_SIMPLEX,
           0.6, (0, 200, 0) if not collision_alert else (0, 0, 255), 2)

        if collision_alert:
            cv2.putText(vis, "COLLISION ALERT!", (10, 80), cv2.
               FONT_HERSHEY_SIMPLEX, 0.8, (0, 0, 255), 3)

        cv2.imshow("optical_flow_collision_pid", vis)

        # ---- Here you would send commands to your flight controller /
           autopilot ----
        # Example (simulation): print or log
        print(f"[{time.strftime('%H:%M:%S')}] {info_text} | CMD: yaw={
           yaw_cmd_s:.2f}, forward={forward_cmd_s:.2f}")

        # Example MAVROS integration stub (uncomment and adapt if using
           ROS / MAVROS):
        # import rospy
        # from geometry_msgs.msg import Twist
        # twist = Twist()
        # twist.linear.x = forward_cmd_s  # forward/back speed
        # twist.angular.z = yaw_cmd_s     # yaw rate
        # vel_pub.publish(twist)

        # handle user keyboard
        key = cv2.waitKey(1) & 0xFF
        if key == 27:  # ESC
            break
        if key == ord('r'):
            pid.reset()

        prev_gray = gray

    cap.release()
    cv2.destroyAllWindows()


# -----------------------
# CLI
# -----------------------
if __name__ == "__main__":
    ap = argparse.ArgumentParser()
    group = ap.add_mutually_exclusive_group(required=False)
    group.add_argument("--video", help="Path to video file (e.g. videos/
       test.mp4)")
    group.add_argument("--camera", help="Camera index (default 0)",
       default="0")
```

```
    ap.add_argument("--show", help="Show display (default true)", action
        ="store_true")
    args = ap.parse_args()
    main(args)
```